慶應義塾 Keio University

DAY 2

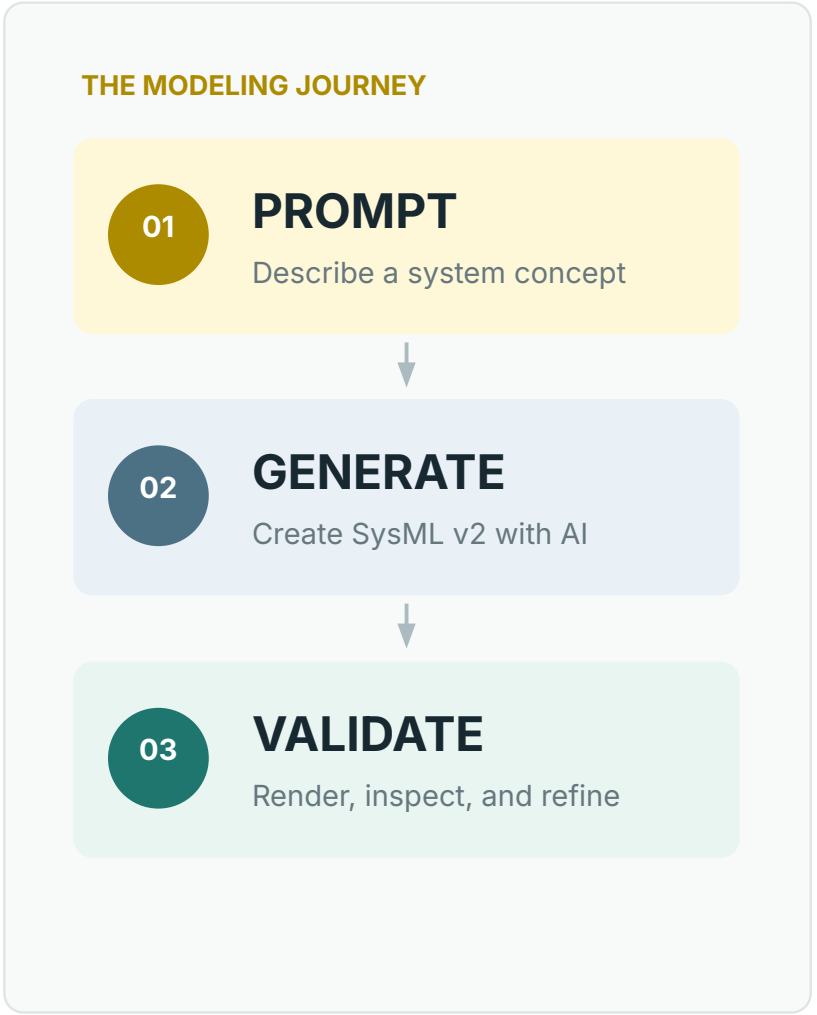
From Prompts to Models

The Model-as-Code Approach

AI-assisted SysML v2 / Agents and harnesses / Jupyter + OpenCode

Laurent Balmelli · Hidekazu Nishimura

Keio University | October 2026



DAY 2 / AGENDA

PERIODS 3 & 4

Class Schedule - Day 2

From AI-assisted SysML modeling to hands-on work with Jupyter and OpenCode.

2.1 How AI can help conceptual product design

90 MIN

- Generate SysML v2 models and display them using views
- Understand LLMs, agents and harnesses through OpenCode
- Evaluate model validity, domain knowledge and human judgment

Break

15 MIN

2.2 Practice with code generation

90 MIN

- Learn Jupyter notebooks and essential OpenSysML commands
- Explore official examples and find inspiration for your own system
- Generate, visualize and refine a model with OpenCode and Jupyter

SECTION 2.1.1 / LLM-BASED SYSML GENERATION

1.5 MIN

From product ideas to formal models

AI can turn a design request into SysML v2 source that engineers can inspect.

01 DESIGN INTENT

“Design a vehicle with four wheels.”

→

02 GENERATIVE AI

The LLM proposes model definitions.

→

03 SYSML V2 SOURCE

The engineer receives inspectable source.

KEY IDEA A generated model is an editable engineering artifact, not merely a diagram.

SECTION 2.1.1 / LLM-BASED SYSML GENERATION

2 MIN

What does generated SysML v2 look like?

A minimal KeioVehicle model starts with parts, definitions, and a single usage.

PROMPT: "Create a vehicle with four wheels. Do not invent propulsion or performance requirements."

KEIOVEHICLE.SYSML

```
package KeioVehicle {
    part def Wheel;
    part def Vehicle {
        part wheels[4] : Wheel;
    }
    part car : Vehicle;
}
```

WHAT THE CODE MEANS

package	A namespace for the example
part def	A reusable system element type
wheels[4]	Exactly four wheel usages
part car	One instance of Vehicle

SECTION 2.1.1 / SYSML V2 VIEWS

3 MIN

A SysML view selects what to display

The model defines the system; the view specifies which elements become visible.

VIEW DEFINITION

```
package KeioViews {
    private import Views::*;
    view completeVehicleModel {
        expose KeioVehicle;
        expose KeioVehicle::**;
    }
}
```

THREE DISTINCT OBJECTS

1. **Model** — KeioVehicle contains definitions
2. **View** — Expose the selected elements
3. **Rendering** — Jupyter draws the chosen view

```
%render KeioViews::completeVehicleModel mermaid
```

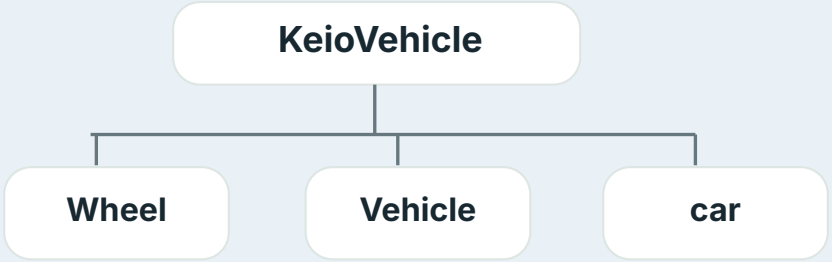
SECTION 2.1.1 / SYSML V2 VIEWS

3 MIN

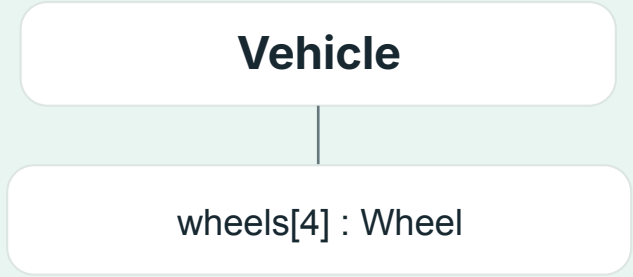
One model can support multiple views

A complete view and a focused view offer different perspectives on the same model.

VIEW A: COMPLETE MODEL



VIEW B: VEHICLE STRUCTURE



FOCUSED VIEW view vehicleStructure { expose KeioVehicle::Vehicle; expose KeioVehicle::Vehicle::**; }

LIVE DEMO · Run both %render commands; did the model change, or only its view?

SECTION 2.1.1 / LLM-BASED SYSML GENERATION

2.5 MIN

What does the research tell us?

LLMs can generate system models, but quality depends on the task and evaluation.

SYSMBENCH / 2025 • Prompt-strategy research / 2026

151

curated scenarios

17

LLMs tested

62%

top reported semantic F1

RESEARCH TAKEAWAY There is no universal winning prompt: provide task context, then inspect and validate.

SECTION 2.1.1 / LLM-BASED SYSML GENERATION

2 MIN

Five ways to prompt the same model

Ask for the same SysML artifact using different types of guidance.

1	ZERO-SHOT	Generate an electric vehicle with four wheels, a battery and motor.
2	FEW-SHOT	Follow: part wheels[4] : Wheel; Add battery and motor similarly.
3	CHAIN-OF-THOUGHT	First outline parts and relations; then generate source and assumptions.
4	CHAIN-OF-VERIFICATION	Draft the model, check definitions, and revise the source.
5	RETRIEVAL / RAG	Consult first_model.md and official examples before extending the model.

Same task: extend KeioVehicle with a battery and motor while preserving four wheels.

SECTION 2.1.1 / CLASS TOOLKIT

2 MIN

OpenCode and Jupyter: one workspace

The native Jupyter AI chat helps students create and examine SysML models.

ONE JUPYTERLAB BROWSER WINDOW

NOTEBOOK / OPENSYSML

```
%load "models/KeioVehicle.sysml"

%render KeioViews::completeVehicleModel mermaid
```

JUPYTER AI CHAT / OPENCODE

“Add a battery and motor; keep the four wheels.”

ACP connects chat to OpenCode; MCP connects OpenCode to notebook tools.

SECTION 2.1.2 / AGENTS AND HARNESSES

3 MIN

From LLMs to agents and harnesses

Three distinct roles make AI-assisted engineering more capable.

01 / AI MODEL

GENERATES

Produces content from a prompt and context.

02 / AI AGENT

PURSUES A GOAL

Chooses actions; responds to tool feedback.

03 / AGENT HARNESS

ENABLES EXECUTION

Provides tools, permissions, context, controls.

Capability comes from the complete system, not the LLM alone.

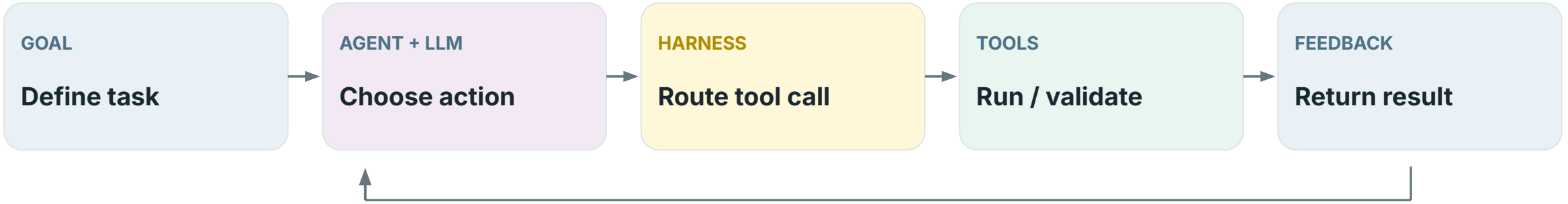
SECTION 2.1.2 / AGENTS AND HARNESSES

3 MIN

What does an agent harness actually do?

A harness connects decisions to tools, execution feedback, and safety controls.

THE AGENT EXECUTION LOOP



WHAT THE HARNESS PROVIDES

Context
Relevant files and instructions

Tools
Editor, shell, validator, MCP

Feedback
Real outputs and errors

Controls
Permissions and stopping rules

The harness makes actions observable; it does not guarantee correct design decisions.

SECTION 2.1.2 / AGENTS AND HARNESSES

3 MIN

OpenCode connects AI to the live notebook

Jupyter AI + ACP + OpenCode + MCP + OpenSysML work as one learning environment.

ONE JUPYTERLAB INTERFACE

NOTEBOOK / OPEN SYSML

```
%render KeioViews::completeVehicleModel mermaid
```

JUPYTER AI CHAT / OPENCODE

“Inspect my current view and run it.”

JUPYTER AI
Chat interface

→

ACP
Agent communication

→

OPENCODE
Agent + harness

→

MCP
Notebook tools

→

OPENSYSML
Kernel + output

READ → EXECUTE → OBSERVE | Edits are also possible through MCP tools.

SECTION 2.1.2 / AGENTS AND HARNESSES

3 MIN

SysMLAgent: generate, check, repair

A grammar checker provides the agent with actionable syntax feedback.

TASK
Natural-language requirement

→

RETRIEVAL
Relevant SysML examples

→

GENERATION
Propose model code

→

CHECKER
ANTLR grammar pass

IF ERROR Return diagnostics to the agent; revise.

IF ACCEPTED Engineer still checks the design.

20 / 20 syntax-valid outputs in the reported study; not proof of physical validity.

SECTION 2.1.2 / AGENTS AND HARNESSES

3 MIN

NEXUS: divide the engineering work

Specialized agents transform industrial documentation into reviewable model artifacts.

01 / EXTRACTION AGENT

PDF → Markdown → structured JSON

Components, interfaces, behavior

→

02 / BUILDING AGENT

JSON → SysML v2

Structure, connections, behavior

→

03 / VERIFICATION AGENT

Check and return diagnostics

Repair feedback and human review

Intermediate JSON makes extraction inspectable before formal code generation.

SECTION 2.1.2 / CLASS TOOLKIT

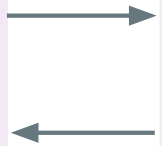
3 MIN

Bringing agent workflows into our toolkit

OpenCode can work across SysML source files and the active Jupyter notebook.

OPENCODE / AGENT HARNESS

Read project context
Propose or edit .sysml
Invoke notebook tools



JUPYTER / OPENSYSML

Load / edit notebook cells
Execute in active kernel
Inspect diagram or errors

IMPORTANT Reference samples remain read-only; source changes need reloading before analysis.

SECTION 2.1.3 / PROGRESS AND LIMITATIONS

2 MIN

Progress: from generation to conformant models

Feedback loops can dramatically improve checker acceptance in a controlled study.

604

test cases

51.16%

first-pass accepted

100%

accepted after repair

WHAT THE RESULT DOES NOT MEAN

Passing a checker is not proof that a physical system is feasible, safe or complete.

SECTION 2.1.3 / PROGRESS AND LIMITATIONS

3 MIN

A valid model is not necessarily a correct design

Language validity, tool conformance, and engineering success are different tests.

1 SYNTAX

Does the language parse?

2 CONFORMANCE

Does the tool accept the model?

3 DESIGN

Does it meet the real requirement?

A four-wheel model can be syntactically valid and still omit an essential energy path.

SECTION 2.1.3 / PROGRESS AND LIMITATIONS

3 MIN

A valid vehicle model can fail its mission

The SysML checker may accept the model while the vehicle misses its range target.

REQUIREMENT: The vehicle must travel at least 300 km per charge.

15 kWh	Battery capacity
20 kWh	Consumption / 100 km

ESTIMATED RANGE **15 ÷ 20 × 100 = 75 km** FAILS 300 KM

Question: at the same consumption, what capacity is needed? Answer: 60 kWh.

SECTION 2.1.3 / PROGRESS AND LIMITATIONS

3 MIN

Next frontier: domain knowledge and human judgment

Knowledge graphs can help identify semantic faults that syntax checks cannot find.

SOURCE MODEL

Syntactically accepted SysML

→

DOMAIN GRAPH

Physical compatibility rules

→

AI PROPOSAL

Candidate localized repair

→

ENGINEER

Review and approve trade-offs

RESEARCH BOUNDARY Strong results on restricted synthetic faults do not prove universal design correctness.

SECTION 2.1.3 / DOMAIN KNOWLEDGE

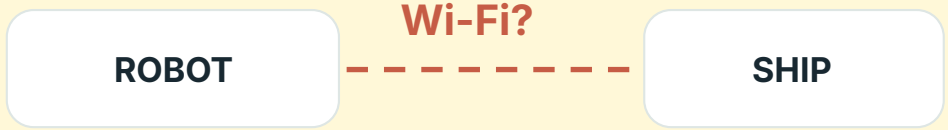
4 MIN

Domain knowledge can expose a hidden mistake

An underwater robot needs more than a formally valid communication link.

MISSION: Stream live HD video from a robot operating 200 m underwater.

AI-GENERATED ARCHITECTURE



Syntactically plausible, physically unsuitable

DOMAIN KNOWLEDGE

Seawater + radio	Very strong attenuation
Acoustic modem	Longer range, limited bandwidth
Fiber-optic tether	High bandwidth, reduced mobility

HUMAN DESIGN CHOICE Use a tether, or record video onboard and transmit later? Decide the trade-off.

SECTION 2.2 / JUPYTER INTRODUCTION

3 MIN

What is a Jupyter notebook?

Code, explanations, and rendered results live together in one interactive document.

EXAMPLE NOTEBOOK / KEIOVEHICLE.IPYNB

MARKDOWN CELL	Vehicle architecture: purpose, assumptions and notes
CODE CELL	%render KeioViews::completeVehicleModel mermaid
OUTPUT	**Vehicle** → wheels[4] : Wheel

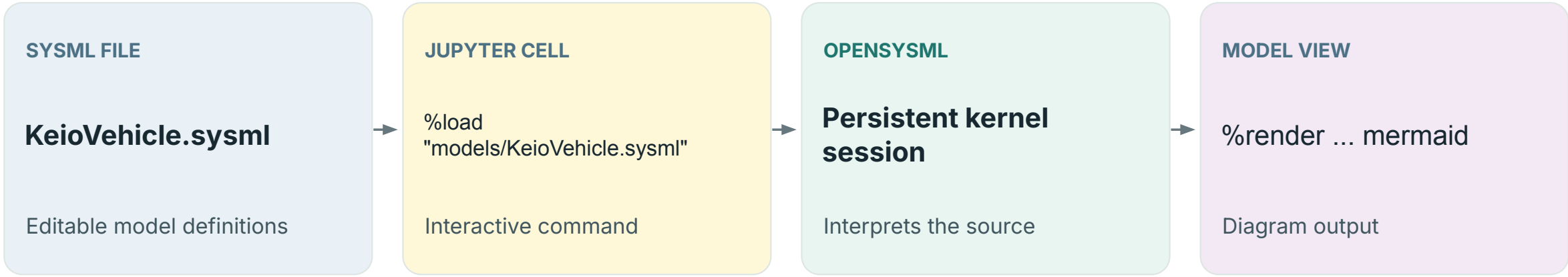
CELL + KERNEL + OUTPUT

SECTION 2.2 / JUPYTER INTRODUCTION

4 MIN

From SysML source to a diagram in Jupyter

Load a source file, execute it in OpenSysML, and render a named view.



REMEMBER: After editing a .sysml file, reload it before rendering the updated model.

A restarted kernel loses in-memory definitions; rerun %load and view cells in order.

SECTION 2.2 / JUPYTER INTRODUCTION

3 MIN

Jupyter quick reference: essential commands

A few keyboard shortcuts and three OpenSysML commands are enough to begin.

JUPYTERLAB / KEYBOARD

Shift + Enter	Run cell; move down
Ctrl + Enter	Run; stay on cell
Esc / Enter	Command / edit mode
A / B	New cell above / below
M / Y	Markdown / code cell
Ctrl + S	Save notebook

OPENSYSML / THREE COMMANDS

%load Load an existing .sysml file

```
%load "models/KeioVehicle.sysml"
```

%print Inspect loaded model elements

```
%print KeioVehicle
```

%render Visualize a named view

```
%render KeioViews::completeVehicleModel mermaid
```

LIVE DEMO · Esc → B → %print KeioVehicle → Shift + Enter

SECTION 2.2 / HANDS-ON MODEL-AS-CODE

10 MIN

Explore the official SysML v2 example models

Use the official vehicle sample to practice loading and rendering a model.

OPENSYSML NOTEBOOK

```
%load ".../SimpleVehicleModel.sysml"

%render OfficialVehicleViews::
vehicleConfigurationA mermaid
```

YOUR TASK

01 **Open official_vehicle_model.md**

02 **Load the official vehicle sample**

03 **Define and render a view**

STUDENT GOAL: Identify three model elements or relationships in the rendered view.

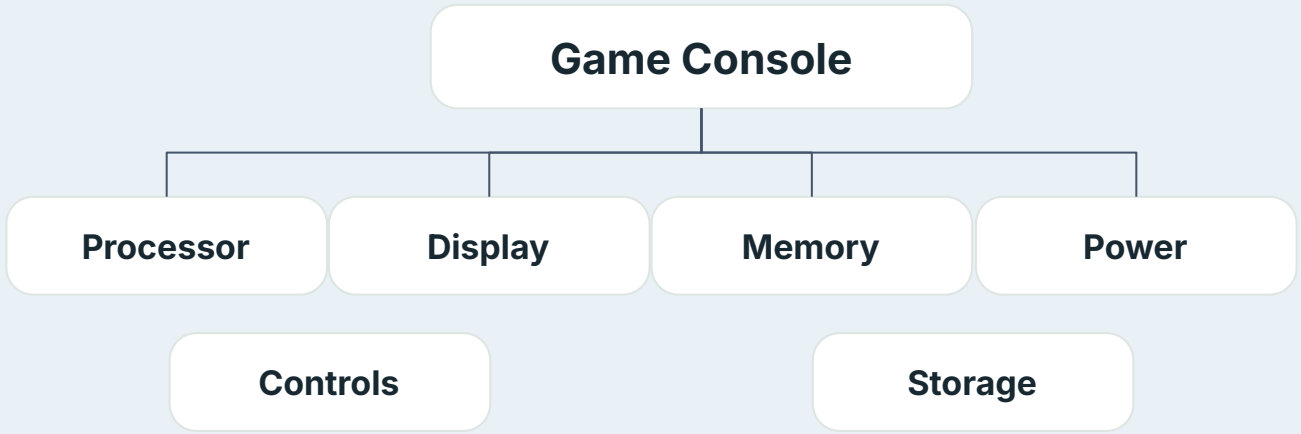
SECTION 2.2 / PRACTICE INSPIRATION

2 MIN

Example A: a video game console

Consumer electronics: model the architecture of a familiar product.

SIMPLE SYSTEM MODEL



MODELING QUESTION

Which elements belong inside the product boundary?

```
part def Controller;  part def GameConsole { part controllers[2] : Controller; }
```

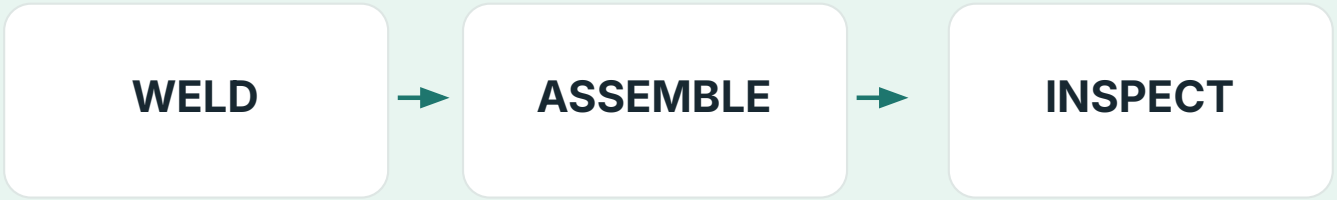
SECTION 2.2 / PRACTICE INSPIRATION

2 MIN

Example B: an automotive assembly factory

An industrial facility is a system made of interacting production systems.

CONCEPTUAL MATERIAL FLOW



MODELING QUESTION

How do we represent nested production systems?

```
part def ProductionLine;  part def CarFactory { part lines[2] : ProductionLine; }
```

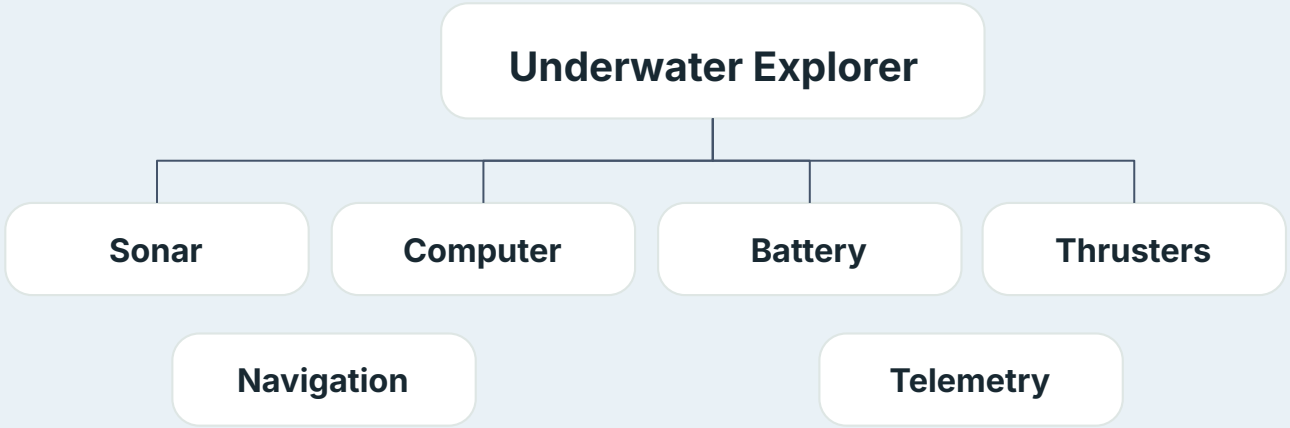
SECTION 2.2 / PRACTICE INSPIRATION

2 MIN

Example C: an autonomous ocean explorer

Embedded systems combine physical hardware, computation, sensing, and control.

CYBER-PHYSICAL ARCHITECTURE



MODELING QUESTION

What must work under harsh environmental limits?

```
part def Thruster;  part def UnderwaterExplorer { part thrusters[2] : Thruster; }
```

SECTION 2.2 / HANDS-ON MODEL-AS-CODE

3 MIN

Choose a system of your own

This small concept becomes the starting point for your individual final report.

01 NAME + PURPOSE

What does your system do?

02 SYSTEM BOUNDARY

What is inside or external?

03 4–8 COMPONENTS

Which subsystems matter first?

04 ONE OPEN QUESTION

What remains uncertain?

Start small: a model you can explain is more valuable than an overbuilt model.

SECTION 2.2 / HANDS-ON MODEL-AS-CODE

12 MIN

Generate a first SysML v2 model with OpenCode

Describe your intended system; let the agent propose a reviewable source file.

OPENCODE / STARTER PROMPT

```
Read the course first model and examples.
Create SysML v2 for my system:
Name: [your system]
Purpose: [one sentence]
Components: [4-8 items]
Use part definitions and usages.
Report assumptions; save a .sysml file.
```

STUDENT WORKFLOW

01 **Tell OpenCode**

02 **Review assumptions**

03 **Save a .sysml file**

OUTPUT: One editable .sysml source file, plus your prompt and assumptions.

SECTION 2.2 / HANDS-ON MODEL-AS-CODE

10 MIN

Load, visualize, inspect, and refine

The agent-generated file is a first draft; examine it using OpenSysML.

OPENSYSML NOTEBOOK

```
%load "models/MySystem.sysml"

package MyViews {
  private import Views::*;
  view overview {
    expose MySystem;
    expose MySystem::**;
  }
}

%render MyViews::overview mermaid
```

INSPECT THE RESULT

✓ **SYNTAX**	Does the tool report errors?
✓ **COMPLETENESS**	What essential parts are missing?
✓ **ASSUMPTIONS**	What did AI invent?
✓ **DESIGN INTENT**	Does this fit your idea?

If the source changes, reload it or restart the kernel and rerun cells in order.

SECTION 2.2 / HANDS-ON MODEL-AS-CODE

4 MIN

Save your first model for the final report

Preserve the model, your visualization, and evidence of your decisions.

MySystem.sysml

Editable formal model

MySystem.ipynb

Notebook and rendered view

prompts.md

AI interactions and decisions

REFLECTION BEFORE YOU LEAVE

01 What did the AI get right?

02 What did you correct, and why?

03 What remains uncertain?

DAY 2 / CLOSING

2 MIN

A generated model is only the beginning

Engineering value comes from understanding, displaying, and challenging the result.

THREE IDEAS TO TAKE AWAY

01 FORMALIZE

Make ideas explicit

Use SysML v2 to define a system, its elements, and its assumptions.

02 VISUALIZE

Select useful views

Use Jupyter and OpenSysML to display and inspect the model.

03 EVALUATE

Keep human judgment

A formally valid model may still be an incorrect design.

FOR YOUR FINAL REPORT

Keep the model, its notebook visualization, and your design decisions.

What did you accept, revise, or reject from the agent's suggestions?